

Cas d'erreur côté serveur

Projet : SAÉ BUT2 S3 2026-2027 **Thème :** Langages de programmation **Responsables :** Mai + Albine **Partie :** Serveur C + Sockets **Date :** 26 septembre 2026

Table des matières

- 1. Objectif
 - 2. Format standard des erreurs
 - 3. Liste complète des codes d'erreur
 - 4. Détail des cas d'erreur
 - 4.1 INVALID_DATA
 - 4.2 USERNAME_TAKEN
 - 4.3 CARD_NOT_FOUND
 - 4.4 CARD_NOT_ACTIVE
 - 4.5 DUPLICATE_CARD
 - 4.6 ALREADY_RECOMMENDED
 - 4.7 NOT_RECOMMENDED
 - 4.8 BATTLE_NOT_FOUND
 - 4.9 BATTLE_ALREADY_ACTIVE
 - 4.10 ALREADY_VOTED
 - 4.11 NOT_LEGITIMATE
 - 4.12 THRESHOLD_NOT_REACHED
 - 4.13 CARDS_NOT_COMPATIBLE
 - 4.14 VA_TOO_LOW
 - 4.15 ALREADY_AUTHENTICATED
 - 4.16 SERVER_BUSY
 - 4.17 INTERNAL_ERROR
 - 5. Gestion côté serveur
 - 5.1 Principe
 - 5.2 Ordre des vérifications
 - 5.3 Logging
 - 5.4 Codes d'erreur par action
 - 6. Correspondance avec le cahier des charges
 - 7. Récapitulatif
 - 8. Validation
-

1. Objectif

Ce document liste les cas d'erreur que le serveur C doit gérer.

Il couvre :

- Les erreurs de validation des données
- Les erreurs de logique métier

- Les erreurs de communication
- Les erreurs internes
- Le comportement du serveur pour chaque erreur

2. Format standard des erreurs

Toutes les erreurs suivent le même format JSON.

```
{
  "status": "ERROR",
  "code": "USERNAME_TAKEN",
  "message": "Ce pseudo est déjà utilise."
}
```

Champ	Type	Description
status	String	Toujours ERROR
code	String	Code machine en majuscules
message	String	Message lisible par l'utilisateur

3. Liste complète des codes d'erreur

#	Code	Signification	Action concernée
1	INVALID_DATA	Champs manquants ou invalides	Toutes
2	USERNAME_TAKEN	Pseudo déjà utilisé	LOGIN
3	CARD_NOT_FOUND	Carte inexistante	RECOMMEND, REPUDIATE, START_BATTLE, VOTE_BATTLE, REQUEST_LEGITIMACY, CLAIM_CARD, AUTHENTICATE_CARD
4	CARD_NOT_ACTIVE	Carte inactive	RECOMMEND, START_BATTLE
5	DUPLICATE_CARD	Carte déjà existante	CREATE_CARD
6	ALREADY_RECOMMENDED	Déjà recommandée	RECOMMEND
7	NOT_RECOMMENDED	Aucune reco à répudier	REPUDIATE
8	BATTLE_NOT_FOUND	Battle inexistante	VOTE_BATTLE

#	Code	Signification	Action concernée
9	BATTLE_ALREADY_ACTIVE	Carte déjà en battle	START_BATTLE
10	ALREADY_VOTED	Déjà voté	VOTE_BATTLE
11	NOT_LEGITIMATE	Utilisateur non légitime	CLAIM_CARD, AUTHENTICATE_CARD
12	THRESHOLD_NOT_REACHED	Seuil non atteint	REQUEST_LEGITIMACY
13	CARDS_NOT_COMPATIBLE	Paradigmes différents	START_BATTLE
14	VA_TOO_LOW	VA inférieure à 5	START_BATTLE
15	ALREADY_AUTHENTICATED	Carte déjà authentifiée	AUTHENTICATE_CARD
16	SERVER_BUSY	Serveur plein	LOGIN
17	INTERNAL_ERROR	Erreur interne du serveur	Toutes

4. Détail des cas d'erreur

4.1 INVALID_DATA

Quand : Un champ obligatoire est manquant ou invalide.

Exemples de déclenchement :

- Créer une carte sans titre
- Créer une carte avec yearCreated = 3000
- Envoyer un JSON mal formé

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "INVALID_DATA",
  "message": "Le titre est obligatoire."
}
```

Comportement serveur :

- Ne mine PAS de bloc
- N'écrit PAS en base de données
- Renvoie l'erreur au client
- Garde les structures en mémoire intactes

4.2 USERNAME_TAKEN

Quand : Un utilisateur tente de se connecter avec un pseudo déjà utilisé.

Exemples de déclenchement :

- usman se connecte
- usman tente de se reconnecter depuis un autre client

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "USERNAME_TAKEN",
  "message": "Ce pseudo est deja utilise."
}
```

Comportement serveur :

- Ne mine PAS de bloc
- N'écrit PAS en base de données
- Renvoie l'erreur au client
- Garde la socket ouverte pour permettre une nouvelle tentative

4.3 CARD_NOT_FOUND

Quand : Une action référence une carte qui n'existe pas.

Exemple de déclenchement :

- Recommander une carte avec cardId = c_999

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "CARD_NOT_FOUND",
  "message": "Carte introuvable."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.4 CARD_NOT_ACTIVE

Quand : Une action est tentée sur une carte inactive.

Exemples de déclenchement :

- Recommander une carte INACTIVE
- Lancer une battle avec une carte INACTIVE

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "CARD_NOT_ACTIVE",
  "message": "Cette carte est inactive."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.5 DUPLICATE_CARD

Quand : Un utilisateur tente de créer une carte avec un titre déjà existant.

Exemple de déclenchement :

- Créer Python alors que Python existe déjà

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "DUPLICATE_CARD",
  "message": "Cette carte existe déjà."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.6 ALREADY_RECOMMENDED

Quand : Un utilisateur tente de recommander une carte qu'il a déjà recommandée.

Exemple de déclenchement :

- usman recommande Python
- usman clique à nouveau sur Recommander

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "ALREADY_RECOMMENDED",
  "message": "Vous avez déjà recommande cette carte."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.7 NOT_RECOMMENDED

Quand : Un utilisateur tente de répudier une recommandation qu'il n'a pas donnée.

Exemple de déclenchement :

- usman clique Repudier sur Python sans l'avoir recommandé

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "NOT_RECOMMENDED",
  "message": "Aucune recommandation a repudier."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.8 BATTLE_NOT_FOUND

Quand : Une action référence une battle qui n'existe pas.

Exemple de déclenchement :

- Voter avec battleId = b_999

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "BATTLE_NOT_FOUND",
  "message": "Battle introuvable."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.9 BATTLE_ALREADY_ACTIVE

Quand : Une carte est déjà engagée dans une autre battle.

Exemple de déclenchement :

- Lancer une battle avec Python alors qu'elle est déjà en battle

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "BATTLE_ALREADY_ACTIVE",
  "message": "Cette carte est deja en battle."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.10 ALREADY_VOTED

Quand : Un utilisateur tente de voter deux fois dans la même battle.

Exemple de déclenchement :

- usman vote pour Python dans la battle b_014

- usman clique à nouveau sur Voter

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "ALREADY_VOTED",
  "message": "Vous avez déjà vote dans cette battle."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.11 NOT_LEGITIMATE

Quand : Un utilisateur non légitime tente une action réservée aux légitimes.

Exemple de déclenchement :

- usman non légitime tente d'authentifier Python

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "NOT_LEGITIMATE",
  "message": "Vous n'etes pas legitime pour cette carte."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.12 THRESHOLD_NOT_REACHED

Quand : Un utilisateur tente de demander la légitimation sans avoir atteint le seuil de 5 recommandations.

Exemple de déclenchement :

- usman a émis 3 recommandations et demande la légitimation

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "THRESHOLD_NOT_REACHED",
  "message": "Vous devez avoir emis au moins 5 recommandations."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.13 CARDS_NOT_COMPATIBLE

Quand : Deux cartes n'ont pas le même paradigme.

Exemple de déclenchement :

- Java (OBJECT) VS Haskell (FUNCTIONAL)

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "CARDS_NOT_COMPATIBLE",
  "message": "Les deux cartes doivent avoir le meme paradigme."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.14 VA_TOO_LOW

Quand : Une carte a une VA inférieure à 5.

Exemple de déclenchement :

- Lancer une battle avec une carte ayant VA = 3

Réponse serveur :

```
{
  "status": "ERROR",
```

```
"code": "VA_TOO_LOW",  
"message": "La carte doit avoir au moins 5 VA."  
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.15 ALREADY_AUTHENTICATED

Quand : Une carte déjà authentifiée est authentifiée à nouveau.

Exemple de déclenchement :

- usman légitime authentifie Python déjà authentifiée

Réponse serveur :

```
{  
  "status": "ERROR",  
  "code": "ALREADY_AUTHENTICATED",  
  "message": "Cette carte est deja authentifiee."  
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
-

4.16 SERVER_BUSY

Quand : Le nombre maximum de clients connectés (10) est atteint.

Exemple de déclenchement :

- 10 clients sont connectés
- Un 11ème tente de se connecter

Réponse serveur :

```
{  
  "status": "ERROR",  
  "code": "SERVER_BUSY",  
}
```

```
"message": "Le serveur est plein. Reessayez plus tard."
}
```

Comportement serveur :

- Refuse la connexion
 - Ferme la socket du nouveau client
 - Logue l'erreur
-

4.17 INTERNAL_ERROR

Quand : Une erreur interne survient côté serveur.

Exemples de déclenchement :

- Erreur de parsing JSON inattendue
- Erreur de base de données

Réponse serveur :

```
{
  "status": "ERROR",
  "code": "INTERNAL_ERROR",
  "message": "Une erreur interne est survenue."
}
```

Comportement serveur :

- Ne mine PAS de bloc
 - N'écrit PAS en base de données
 - Renvoie l'erreur au client
 - Logue l'erreur pour analyse
-

5. Gestion côté serveur

5.1 Principe

Quand une action échoue, le serveur :

1. N'ajoute PAS de bloc à la blockchain
2. N'écrit PAS en base de données
3. Renvoie un message d'erreur au client
4. Garde les structures en mémoire intactes

5.2 Ordre des vérifications

Le serveur vérifie dans cet ordre :

1. Format du JSON
2. Champs obligatoires
3. Existence de l'objet (carte, utilisateur, battle)
4. Statut de l'objet (ACTIVE, INACTIVE)
5. Règles métier (VA, paradigme, seuil)
6. Autorisations (légitime, propriétaire)

5.3 Logging

Le serveur log toutes les erreurs dans la console.

```
fprintf(stderr, "[ERROR] %s : %s\n", code, message);
```

5.4 Codes d'erreur par action

LOGIN :

- USERNAME_TAKEN
- SERVER_BUSY
- INVALID_DATA

CREATE_CARD :

- INVALID_DATA
- DUPLICATE_CARD
- INTERNAL_ERROR

RECOMMEND :

- CARD_NOT_FOUND
- CARD_NOT_ACTIVE
- ALREADY_RECOMMENDED

REPUDIATE :

- CARD_NOT_FOUND
- NOT_RECOMMENDED

START_BATTLE :

- CARD_NOT_FOUND
- CARD_NOT_ACTIVE
- CARDS_NOT_COMPATIBLE
- VA_TOO_LOW
- BATTLE_ALREADY_ACTIVE

VOTE_BATTLE :

- BATTLE_NOT_FOUND
- ALREADY_VOTED

- NOT_ALLOWED

REQUEST_LEGITIMACY :

- CARD_NOT_FOUND
- THRESHOLD_NOT_REACHED

CLAIM_CARD :

- CARD_NOT_FOUND
- NOT_LEGITIMATE

AUTHENTICATE_CARD :

- CARD_NOT_FOUND
- NOT_LEGITIMATE
- ALREADY_AUTHENTICATED

6. Correspondance avec le cahier des charges

Cas d'erreur	Cas du cahier des charges
INVALID_DATA	Cas 8, 11
USERNAME_TAKEN	Cas 4, 5, 7
CARD_NOT_FOUND	Cas 12, 14, 15
CARD_NOT_ACTIVE	Cas 12, 15, 21
DUPLICATE_CARD	Cas 11
ALREADY_RECOMMENDED	Cas 12
NOT_RECOMMENDED	Cas 14
BATTLE_NOT_FOUND	Cas 16
BATTLE_ALREADY_ACTIVE	Cas 15
ALREADY_VOTED	Cas 16
NOT_LEGITIMATE	Cas 19, 20
THRESHOLD_NOT_REACHED	Cas 18
CARDS_NOT_COMPATIBLE	Cas 15
VA_TOO_LOW	Cas 15
ALREADY_AUTHENTICATED	Cas 20
SERVER_BUSY	Cas 4
INTERNAL_ERROR	Cas 8

7. Récapitulatif

Catégorie	Nombre
Erreurs serveur	17
Actions concernées	11
Codes d'erreur	17

8. Validation

Rôle	Nom	Statut
Serveur C	Mai	En attente
Serveur C	Albine	En attente
Client Java	Usman	En attente
Blockchain + BDD	Iyore	En attente
